Start Date : 7 July 2025
LIVE Session Time : (Mon-Fri) 9:00PM IST
Session Duration : 1 Hours
Batch Duration : 3-4 Months
Content Access : 3 Years

⭐ Key Features ⭐

- Beginner Friendly
- Basics to Advanced
- LIVE Sessions (Theory + Practical)
- Session Recordings
- Certification
- 10+ Libraries
- 15 Modules
- 20+ Projects
- 50+ Live Sessions
- 70+ Problem Statements
- 500+ Interview Questions

📚 Syllabus 📚

✅ Python Programming Foundations

📕 Python Syntax and Structure

- Installing Python and setting up environment
- Writing your first Python script
- Python indentation rules
- Comments and docstrings
- Running scripts via command line and IDE

📕 Variables, Data Types, and Type Casting

- Variable declaration and dynamic typing
- Primitive data types:
  - Integers, floats, booleans, strings
- Composite data types:
  - Lists, tuples, dictionaries, sets
- Mutable vs immutable types
- Type casting and type() function

📕 Control Flow (Conditionals and Loops)

- if, elif, and else statements
- Nested conditionals
- Boolean logic and comparison operators
- for and while loops
- Loop control statements: break, continue, pass
- Using range() effectively

📕 Functions and Recursion Basics

- Defining and calling functions
- Parameters and return values
- Default, keyword, and variable-length arguments (*args, **kwargs)
- Scope: local vs global variables
- Recursion fundamentals and base case identification
- Simple recursion examples: factorial, Fibonacci, sum of digits

📕 Modules and Packages

- Importing built-in modules (math, random, sys, datetime)
- Creating and importing custom modules
- Exploring dir() and help() functions
- Using from ... import ... syntax
- Intro to Python Package Index (PyPI) and installing packages with pip

📕 Exception Handling

- What are exceptions and why they matter
- Using try, except, finally blocks
- Catching specific exceptions (ZeroDivisionError, ValueError, etc.)
- Raising custom exceptions with raise

📕 File Handling

- What are Files
- Creating files in different modes
- Opening and closing of files
- Modes : x, a, r, w
- Read and write operation on files
- Deleting a created file

🎁 5 Problem Statements

✅ Object-Oriented Programming in Python

📕 Classes and Objects

- What is OOP and why use it
- Defining a class using class
- Creating objects (instances of a class)
- __init__() method (constructor)
- self keyword and instance variables
- Class vs instance attributes
- Accessing object attributes and methods

🟥 Methods in Classes

- Instance methods
- Class methods (using @classmethod)
- Static methods (using @staticmethod)
- Differences between instance, class, and static methods
- When and why to use each type

🟥 Encapsulation and Data Hiding

- Access modifiers: public, protected , private
- Getters and setters
- Property decorators (@property, @<property>.setter)
- Hiding internal object details
- Real-world encapsulation examples

🟥 Inheritance

- Single inheritance: base and derived classes
- Method overriding
- Using super() to call parent methods
- Multi-level and hierarchical inheritance
- Python's support for multiple inheritance
- MRO (Method Resolution Order)

🟥 Polymorphism

- Method overriding
- Operator overloading (__add__, __eq__, etc.)
- Duck typing in Python
- Function overloading workaround using *args and **kwargs
- Real-world polymorphism examples

🟥 Abstraction

- Hiding implementation details
- Abstract base classes (ABC module)
- Using @abstractmethod
- Designing interfaces and base templates

🟥 Special/Magic Methods

- Constructor and destructor: __init__(), __del__()
- String representation: __str__(), __repr__()
- Operator overloading: __add__(), __sub__(), __mul__(), __eq__(), etc.
- Object comparison: __lt__(), __gt__(), __le__(), __ge__()
- Callable objects: __call__()

🟥 Composition vs Inheritance

- Composition: "has-a" relationship
- Inheritance: "is-a" relationship
- Choosing the right design principle
- Building larger classes using composition

🎁 5 Problem Statements

✅ Popular Python libraries

🎯 collections

🎯 heapq

🎯 bisect

🎯 itertools

🎯 array

🎯 math

🎯 queue

🎯 pandas

🎯 random

🎯 numpy

🎯 matplotlib

🎁 5 Problem Statements

✅ **Time and Space Complexity**

📕 What is Time Complexity?

- Introduction to Big-O notation
- Best, average, and worst-case analysis
- Order of growth: Constant < Logarithmic < Linear < Linearithmic < Quadratic < Exponential
- Importance in algorithm selection and problem scaling

📕 Common Time Complexities

- O(1) – Constant time
- O(log n) – Binary search, divide-and-conquer steps
- O(n) – Linear scans, loops
- O(n log n) – Merge sort, heap sort, quick sort (avg)
- $O(n^2)$, $O(n^3)$ – Nested loops, brute force
- O(2^n), O(n!) – exhaustive problems

📕 Analyzing Loops and Recursion

- Single and nested loops
- Logarithmic loops (i *= 2, etc.)
- Multiple loops vs nested loops
- Recursion trees:
  - Fibonacci
  - Binary search recursion
  - Merge sort (T(n) = 2T(n/2) + O(n))
- Master Theorem basics

📕 What is Space Complexity?

- Memory usage of algorithms
- Primitive data types vs collections
- Stack space for recursion
- Auxiliary vs input space
- In-place algorithms

📕 Common Space Complexities

- O(1) – Constant memory (in-place operations)
- O(n) – Arrays, hash maps
- O(log n) – In recursive binary search
- $O(n^2)$ – 2D matrices, DP tables

📕 Time and Space Complexity of Key Operations

- Arrays: access, insert, delete
- Lists, dictionaries, sets in Python

- Stack and Queue operations
- Binary Search: O(log n) time, O(1) space
- Hash map: Average O(1), Worst-case O(n)

🎁 5 Problem Statements

✅ Arrays and Strings

📕 Arrays / Lists in Python

- Python lists as dynamic arrays
- Creating and initializing lists
- Indexing and slicing
- List methods:
  - append(), extend(), insert(), remove(), pop()
  - index(), count(), sort(), reverse(), copy()
- List comprehensions
- Nested lists and 2D arrays
- Time and space complexity of list operations

📕 Common Array Operations

- Traversing arrays (for/while loops)
- Searching:
  - Linear search
  - Binary search (sorted arrays only)
- Insertion and deletion at specific positions
- Reversing an array
- Rotating arrays (left/right rotation)

📕 Two-Pointer Technique

- Concept and use cases
- Finding pairs with a given sum
- Removing duplicates from sorted array
- Partitioning arrays (even-odd, positive-negative)
- Merging two sorted arrays in-place

📕 Sliding Window Technique

- Fixed-size sliding window:
  - Maximum/minimum sum subarray
- Variable-size window:
  - Longest substring/array with unique characters
  - Longest subarray with sum ≤ K
- Optimizing brute force approaches using sliding window

📕 String Fundamentals

- Strings in Python are immutable
- Creating and accessing strings
- Slicing and indexing
- String methods:
  - lower(), upper(), strip(), split(), join(), replace()
- String formatting:
  - f-strings
  - .format() method

📕 String Pattern Problems

- Reversing words and characters
- Checking for:
  - Palindrome
  - Anagram

    - Substring
- Character frequency count using dictionaries
- Removing duplicates

🎁 5 Problem Statements

✅ Linked Lists, Stacks, and Queues

📕 Singly Linked List (SLL)

- Node and list class structure
- Creating nodes and linking
- Traversal and printing nodes
- Insertion:
    - At head
    - At tail
    - At specific position
- Deletion:
    - From head
    - From tail
    - By value
- Searching in a linked list
- Time and space complexity of operations

📕 Doubly Linked List (DLL)

- Node structure with previous and next pointers
- Creating and linking nodes
- Insertion at head, tail, and specific index
- Deletion from head, tail, and specific index
- Traversing forward and backward
- Comparison with singly linked lists

📕 Circular Linked List (CLL)

- Concept and use cases
- Singly vs doubly circular lists
- Insertion and deletion logic
- Traversal and loop stopping condition

📕 Detecting and Removing Cycles

- Floyd's Cycle Detection Algorithm
- Detecting the start of the loop
- Removing loop from a linked list

📕 Merging Linked Lists

- Merging two sorted linked lists
- Merge k sorted linked lists
- Finding intersection node
- Finding middle of a linked list (slow-fast pointers)

📕 Stack – Last In First Out (LIFO)

- Implementing stack using list, deque, or custom class
- Stack operations: push, pop, peek, isEmpty
- Stack use-cases:
    - Expression evaluation (infix/postfix)
    - Parentheses matching

📕 Queue – First In First Out (FIFO)

- Queue implementation using list, deque, and custom class
- Enqueue and dequeue operations

- Queue use-cases:
  - Scheduling, buffering, level-order traversal
- Types of queues:
  - Simple queue
  - Circular queue
  - Priority queue (using heapq)
  - Double-ended queue (deque)

🟥 Deque – Double-Ended Queue

- Insert/delete from both ends
- Python collections.deque usage
- Applications:
  - Sliding window maximum
  - Palindrome checking

🎁 5 Problem Statements

✅ Pointers

🟥 Basics of Pointers

- Pointer declaration and initialization
- Address-of operator &
- Dereferencing operator *
- Null pointers
- Dangling and wild pointers

🟥 Pointer Arithmetic

- Increment and decrement
- Pointer comparisons
- Accessing array elements using pointers
- Pointer to pointer (**ptr)

🟥 Pointers and Functions

- Call by value vs call by reference
- Returning multiple values via pointers
- Swapping using pointers
- Function pointers
- Passing functions as parameters

🟥 Slow and Fast Pointer Techniques

- Cycle detection
- Find middle of linked list
- Detect intersection in two linked lists
- Remove N-th node from end

🟥 Circular and Double Pointers

- Circular linked list
- Doubly linked list

🎁 5 Problem Statements

✅ Trees

🟥 Introduction to Trees

- What is a tree? Definitions and properties
- Terminology:

- Root, child, parent, leaf, sibling, height, depth, degree
- Tree vs Binary Tree vs Binary Search Tree
- Types of trees :
  - N-ary tree, Binary tree, Full/Complete/Perfect trees

🟥 Binary Tree Fundamentals

- Structure of a binary tree node (left, right, data)
- Creating a binary tree
- Basic tree construction using input lists or arrays
- Tree traversal overview:
  - Inorder (Left, Root, Right)
  - Preorder (Root, Left, Right)
  - Postorder (Left, Right, Root)
  - Level Order (BFS using Queue)

🟥 Binary Search Tree (BST)

- Definition and properties
- Inserting a node into a BST
- Searching for a node in a BST
- Deleting a node from a BST
  - Deleting leaf, node with one child, node with two children
- Inorder traversal gives sorted order
- Finding min/max element in BST

🟥 Balanced Trees and Height Concepts

- Concept of balance in a tree
- Balanced vs Unbalanced trees
- Height-balanced check (O(n) approach)
- Skewed trees and their drawbacks

🟥 AVL Tree (Self-Balancing BST)

- Definition and balance factor
- Rotations:
  - Right rotation (LL)
  - Left rotation (RR)
  - Left-Right (LR)
  - Right-Left (RL)
- Insertion with balancing
- Deletion and rebalancing
- AVL tree vs BST complexity analysis

🟥 Tree Construction Techniques

- Build tree from preorder and inorder/postorder traversal
- Serialize and deserialize binary tree
  - BFS and DFS approaches
- Convert sorted array to balanced BST

🟥 Level Order & Variants

- Level order traversal using queue
- Zigzag level order
- Bottom-up level order
- Vertical order traversal
- Diagonal traversal
- Left and right view of binary tree
- Top view and bottom view

🎁 5 Problem Statements

✅ Advanced Trees & Hashing

🟥 Trie

- What is a Trie and when to use it
- Structure of a Trie node
- Inserting a word into a Trie
- Searching and prefix checking
- Deleting a word from a Trie
- Space vs time tradeoffs in Trie vs HashMap

🟥 Heap – Min Heap / Max Heap

- What is a heap
- Min heap vs Max heap
- Heap implementation using array
- Building a heap from an array
- Insertion and deletion (heapify up/down)
- Heap operations:
  - insert(), extract_min(), heapify()
- Python's heapq module (min heap by default)
- Applications:
  - Priority queue
  - Top-K elements
  - K largest/smallest elements

🟥 Segment Tree

- What is a segment tree and its use case
- Tree construction for:
  - Range sum queries
  - Range minimum/maximum queries
- Querying a segment tree
- Updating a segment tree
- Time and space complexity: O(log n) per operation

🟥 Hashing Basics

- What is hashing and why it's important
- Hash functions and collision handling
- Time and space complexity
- Python dictionaries (dict) and set as hash tables
- Common operations and performance characteristics

🟥 Collision Resolution Techniques

- Open addressing:
  - Linear probing
  - Quadratic probing
  - Double hashing
- Separate chaining with linked lists

🟥 Applications of Hashing

- Frequency counting (most frequent elements)
- Detecting duplicates
- Longest consecutive sequence

🎁 5 Problem Statements

✅ Graphs

🟥 Introduction to Graphs

- What is a graph? Definitions and terminology

  - Vertex, edge, degree, path, cycle, connected, components
- Types of graphs:
  - Directed vs Undirected
  - Weighted vs Unweighted
  - Cyclic vs Acyclic
- Real-world use cases: networks, maps, dependency resolution.

Graph Representations

- Adjacency Matrix:
  - Space complexity: $O(V^2)$
  - Good for dense graphs
- Adjacency List:
  - Space complexity: $O(V + E)$
  - Good for sparse graphs
- Edge list representation
- Converting between representations
- Python implementation using:
  - Lists
  - Dictionaries (defaultdict)
  - collections.defaultdict(list)

Depth First Search (DFS)

- Recursive DFS implementation
- Iterative DFS using stack
- Use cases:
  - Detecting cycles
  - Topological sort
  - Path finding
- Marking visited nodes
- DFS time complexity

Breadth First Search (BFS)

- BFS using queue
- Level-order traversal (in unweighted graphs)
- Applications:
  - Shortest path in unweighted graphs
  - Checking bipartite graphs
  - Graph connectivity
- BFS vs DFS comparison
- BFS time complexity

Cycle Detection

- In undirected graphs:
  - DFS with parent tracking
- In directed graphs:
  - DFS with recursion stack
- Cycle detection use cases:
  - Deadlock detection
  - Validating DAGs

Topological Sorting

- Basics of topological sort
- Directed Acyclic Graphs (DAG)
- Kahn's Algorithm
- DFS-based topological sort
- Time complexity

Graph Traversal Edge Cases

- Handling disconnected graphs

- Graphs with self-loops or multiple edges
- Dealing with infinite cycles
- Graphs with multiple components

🟥 Graph Input and Parsing

- Reading graph data from:
  - Edge list
  - Adjacency list
  - Matrix input
- Mapping node names to numbers

🟥 Union-Find / Disjoint Set Union (DSU)

- What is Union-Find and when to use it
- Key operations:
  - find(): with path compression
  - union(): with union by rank or size
- Applications:
  - Cycle detection
  - Connected components
  - Kruskal's algorithm
- Python implementation using lists

🟥 Minimum Spanning Tree (MST)

- Prim's Algorithm :
  - Greedy approach
  - Min-heap + visited set
  - For dense graphs
- Kruskal's Algorithm :
  - Greedy edge-based approach
  - Sorting edges by weight
- Time complexity comparison

🟥 Shortest Path Algorithms

- Dijkstra's Algorithm :
  - Priority queue (min-heap)
  - Non-negative edge weights
  - Time complexity
- Bellman-Ford Algorithm :
  - Negative weights
  - Detecting negative cycles
  - Relaxing all edges (V-1) times
- Floyd-Warshall Algorithm :
  - All-pairs shortest paths
  - Matrix-based DP approach
  - Time complexity

🟥 Grid-Based Graph Algorithms

- What is Grid
- Converting 2D grids into graphs
- Graph traversal on grids (DFS/BFS)

🟥 Strongly Connected Components (SCC)

- What are SCC
- Kosaraju's Algorithm
- Tarjan's Algorithm
- Time Complexities

🟥 Bridges and Articulation Points

- What is Bridge

- What is Articulation Point
- Use cases:
  - Network reliability, communication systems

📕 Eulerian Path & Circuit

- Eulerian Path:
  - Visits every edge exactly once
- Eulerian Circuit:
  - Eulerian path that starts and ends at same node
- Conditions for existence
- Fleury's algorithm (for undirected graphs)

🎁 5 Problem Statements

✅ Sorting and Searching

📕 Introduction to Sorting

- Why sorting is important
- Stable vs unstable sorts
- In-place vs out-of-place sorting
- Time and space complexity comparisons
- Best, average, and worst-case performance

📕 Basic Sorting Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Time and space complexity comparisons

📕 Divide and Conquer Sorting

- Merge Sort :
  - Divide and merge logic
  - Recursive and iterative implementations
  - Space complexity: O(n)
- Quick Sort :
  - Pivot-based partitioning
  - Average case: O(n log n), Worst case: $O(n^2)$
  - In-place but not stable

📕 Heap Sort

- Max-Heap based sorting
- Building heap from array
- Sorting in descending order with min-heap
- Time complexity: O(n log n)
- In-place but not stable
- Compare with quick sort in practice

📕 Non-Comparison Based Sorting

- Counting Sort :
  - for small range of integers
  - Stable and fast: O(n + k)
- Radix Sort :
  - Sorting by digits (LSD/MSD)
  - Use of counting sort internally
- Bucket Sort

📕 Linear Search

- Basic element search

- First/last/any occurrence
- Linear time complexity: O(n)

🟥 Binary Search (Fundamentals)

- Sorted array basics
- Iterative and recursive implementations
- Time complexity: O(log n)
- Common errors: overflow, mid-point handling
- Python implementation with loops and recursion

🟥 Advanced Binary Search Variants

- Lower bound / upper bound
- Finding first/last occurrence of a number
- Peak element in a mountain array

🎁 5 Problem Statements

✅ Recursion and Backtracking

🟥 Introduction to Recursion

- What is recursion?
- Base case and recursive case
- Call stack behavior in recursion
- Stack overflow and infinite recursion
- Tracing recursive calls manually
- Recursive vs iterative approaches

🟥 Classic Recursion Problems

- Factorial of a number
- Fibonacci series
- Sum of natural numbers
- Reverse a string
- Count number of digits

🟥 Recursion on Arrays and Strings

- Linear recursion on arrays
- Recursive search in an array
- Finding max/min in array recursively
- Reversing arrays using recursion
- Recursion on strings

🟥 Recursion Tree and Time Complexity

- Visualizing recursion using trees
- Analyzing time complexity of recursive calls
- Identifying overlapping subproblems

🟥 Introduction to Backtracking

- What is Backtracking
- Backtracking vs brute-force vs recursion
- State management and pruning

🟥 Classic Backtracking Problems

- N-Queens Problem :
  - Placing N queens without attacks
  - Board representation and pruning
- Rat in a Maze :
  - Path finding in a 2D grid

    - Marking visited and backtracking
- Sudoku Solver :
    - Backtracking with constraints
    - Fast checking of rows, cols, boxes
- Word Search in Grid :
    - DFS + backtracking with visited set

📕 Subsets and Permutations

- Subsets generation :
    - With and without duplicates
- Permutations :
    - String/array permutations
    - Unique permutations (with hashset)
- Combination Sum Problems :
    - Allowing/excluding repetitions
    - K-combinations from array

📕 Optimization in Backtracking

- Pruning invalid branches early
- Sorting input for pruning (combo sum)

🎁 5 Problem Statements

✅ Dynamic Programming

📕 Introduction to Dynamic Programming

- What is Dynamic Programming (DP)?
- Characteristics of DP problems:
    - Overlapping subproblems
    - Optimal substructure
- Difference between:
    - Recursion
    - Memoization (top-down DP)
    - Tabulation (bottom-up DP)
- When to use DP instead of recursion or greedy

📕 Memoization (Top-Down)

- Concept of storing recursive results
- Using dictionaries or arrays to cache results
- Avoiding repeated work in recursion
- Template for memoization
- Python tools:
    - Manual memoization
    - functools.lru_cache

📕 Tabulation (Bottom-Up)

- Iterative approach to solving DP
- Creating and filling a DP table
- Converting recursion to tabulation
- Space optimization:
    - From 2D to 1D arrays
    - Reusing previous states

📕 1D Dynamic Programming Patterns

- Fibonacci Number
- Climbing Stairs
- Minimum Cost Climbing Stairs
- House Robber

- Jump Game

🟥 2D Dynamic Programming Basics

- Grid Path Problems :
  - Unique paths (only down/right)
  - Unique paths with obstacles
- Min path sum in a grid :
  - From (0,0) to (n-1, m-1)
- Visualizing 2D DP with tables and directions

🟥 String DP Problems

- Longest Common Subsequence (LCS) :
- Longest Common Substring
- Edit Distance / Levenshtein Distance
  - Operations (insert, delete, replace)
- Palindrome Substrings

🟥 0/1 Knapsack Problem

- Intro to knapsack:
  - Given weight and value arrays
- Recursive → memoized → tabulated versions
- Related problems:
  - Subset Sum
  - Count subsets with given sum

🟥 Matrix Chain Multiplication (MCM)

- Parenthesization problem
- Minimum number of scalar multiplications
- Recursive structure and DP table formulation
- Memoization and tabulation approaches

🟥 DP on Subsequences (Inclusion-Exclusion Style)

- Subset Sum II (count number of subsets)
- Target Sum (with + and - operations)
- K Sum Problem

🟥 DP on Strings

- Longest Palindromic Subsequence (LPS)
- Minimum insertions/deletions to convert one string to another
- Shortest Common Supersequence

🟥 DP on Trees

- DFS + DP on tree structure
- Tree DP states and transition idea
- DP for diameter of a tree

🟥 Bitmask Dynamic Programming

- What is Bitmask DP
- Why Bitmask DP
- State representation via bits
- Travelling Salesman Problem (TSP)

🎁 5 Problem Statements

✅ Bit Manipulation

🟥 Binary Representation Basics

- Converting numbers between binary and decimal
- Signed vs unsigned binary (two's complement)
- Number of bits in integers (e.g., 32-bit, 64-bit)

🟥 Bitwise Operators

- AND &
- OR |
- XOR ^
- NOT ~
- Left Shift
- Right Shift

🟥 Basic Bit Operations

- Set a bit
- Clear a bit
- Toggle a bit
- Check if bit is set

🟥 Bit Tricks

- Count set bits
- Check if power of 2
- Turn off rightmost set bit
- Isolate rightmost set bit
- Swap two numbers without temp

🟥 Bit Masks

- Creating and applying masks
- Extracting specific bits
- Turning off bits in a range

🟥 Bit Manipulation in Arrays

- Using XOR for frequency
- Cumulative XOR (Prefix XOR)
- Bitwise representation of sets

🟥 Problem Patterns

- Single Number (I, II, III)
- Missing number in sequence
- Count total set bits from 1 to N
- Bitwise AND/OR over ranges
- Minimum flips to convert A to B

✅ Interview Questions

🟥 500+ Interview Questions

Book Your Seat Now : https://www.5minutesengineering.com

In case of any query/doubt/difficulty regarding this batch, directly message me on whatsapp 8329917036

Or mail me at 5minutesengineering@gmail.com

~~₹9999~~

MEGA SAVE OFFER

70% OFF

₹2999

WANT MORE DISCOUNT

USE CODE

MEGASAVE

To Get ₹500 OFF

₹2499

www.5minutesengineering.com